

NO-HALU MODEL

TECHNISCH IMPLEMENTATIEDOCUMENT

Architectuur — API-specificaties — Dataflow — Implementatiegids

Versie 1.0 — 2026

Definitie: No-Halu

De naam 'No-Halu' staat voor 'Low Hallucination'. Het systeem is ontworpen om hallucinaties in AI-uitkomsten maximaal te reduceren. Hallucinaties kunnen echter nooit volledig worden uitgesloten. Het No-Halu Model minimaliseert het risico aanzienlijk, maar vervangt geen inhoudelijke domeinexpertise of menselijke verificatie.

1. Systeemoverzicht

Het No-Halu Model (NHM) is een gedistribueerde multi-agent inferentiearchitectuur ontworpen voor productie-omgevingen. Het systeem orchestreert drie parallelle LLM API-aanroepen, evalueert de resulterende responsen via een scoringsalgoritme, voert kruisvalidatie uit en convergeert via iteratieve harmonisatie naar een betrouwbare uitkomst.

Component	Beschrijving
Orchestrator	Centrale controller: beheert agentcycli, aggregeert scores, coördineert harmonisatie
Agent Pool	Drie geïsoleerde LLM-instanties (A1, A2, A3) met onafhankelijke context-windows
Scoring Engine	Berekent S(R) op vier dimensies: consistentie, factualiteit, coherentie, brongrounding
Verifier Module	Adversariale kruisverificatie: verliezende agents challengen de winnaar
Harmonizer	Extractie en integratie van superieure componenten uit alle responsen
Convergence Monitor	Bewaakt divergentiemetric $D(n)$ en bepaalt halt-condities
Cache Layer	Redis-gebaseerde caching van tussenliggende responsen voor efficiëntie

2. API-Specificaties

2.1 Primaire Endpoint

```
POST /api/v1/nhm/query
```

Request Body (JSON):

```
{
  "prompt":      string,    // De gebruikersprompt (max 8192 tokens)
  "domain":      string,    // Optioneel: "medical"|"legal"|"technical"|"general"
  "max_iter":    integer,   // Max iteraties (standaard: 10)
  "threshold":   float,     // Convergentiedrempel epsilon (standaard: 0.05)
  "weights": {
    "consistency": float,   // alpha (standaard: 0.30)
    "factuality":  float,   // beta  (standaard: 0.30)
    "coherence":   float,   // gamma (standaard: 0.25)
    "grounding":   float    // delta (standaard: 0.15)
  }
}
```

Response Body (JSON):

```
{
  "result":      string,    // De geharmoniseerde eindrespons
  "confidence":  float,     // Vertrouwensscore (0.0-1.0)
  "iterations":  integer,   // Aantal voltooide iteraties
  "converged":   boolean,   // True als convergentiecriterium bereikt
  "agent_scores": object,   // Individuele scores per agent
  "divergence_trace": array, // D(n) per iteratie
  "processing_ms": integer   // Totale verwerkingstijd in ms
}
```

2.2 Streaming Endpoint

```
POST /api/v1/nhm/query/stream
```

Server-Sent Events (SSE) stream met de volgende event-types:

- `agent_response`: Tussentijdse agentrespons beschikbaar.
- `score_update`: Scorings-evaluatie voltooid.
- `verification_start`: Kruisverificatie begonnen.
- `harmonization`: Harmonisatieronde voltooid.
- `complete`: Finale geconvergeerde respons beschikbaar.
- `error`: Foutmelding met details.

3. Dataflow en Procesarchitectuur

3.1 Fasenoverzicht

Fase	Naam	Beschrijving	Duur (est.)
0	Preprocessing	Prompt sanitatie, domeindetectie, gewichtskalibratie	~10ms
1	Parallel Inference	Gelijktijdige aanroep van A1, A2, A3 via async I/O	~800ms
2	Scoring	S(R) berekening voor elke respons via Scoring Engine	~50ms
3	Winnaar-selectie	$\text{argmax}(S(R_i))$ bepaalt de winnende agent A^*	~5ms
4	Kruisverificatie	A1, A2 genereren weerleggingsscore $W(A_i, A^*)$	~600ms
5	Harmonisatie	merge() integreert superieure componenten	~200ms
6	Convergentiecheck	$D(n) < \epsilon$? Halt of nieuwe iteratiecyclus	~10ms
7	Postprocessing	Formatting, metadata-annotatie, cache-schrijven	~20ms

3.2 Scoring Engine Implementatie

```
class ScoringEngine {
  public function calculate(string $response, string $prompt): float {
    // Gewogen score op vier dimensies
    $consistency = $this->measureConsistency($response);
    $factuality  = $this->measureFactuality($response);
    $coherence   = $this->measureCoherence($response, $prompt);
    $grounding   = $this->measureGrounding($response);

    return ($this->alpha * $consistency)
      + ($this->beta * $factuality)
      + ($this->gamma * $coherence)
      + ($this->delta * $grounding);
  }
}
```

3.3 Kruisverificatie Algoritme (pseudocode)

```
FUNCTION cross_verify(winner, others):
  FOR EACH agent IN others:
    challenger = agent.challenge(winner)
    W[agent] = 1 - cosine_similarity(embed(challenger), embed(winner))
  RETURN max(W.values())
```

OPMERKING: Cosinus-similariteit wordt berekend via een lokaal embedding-model om API-kosten en latentie te minimaliseren.

4. Infrastructuurvereisten

4.1 Minimale Systeemvereisten

Component	Minimum	Aanbevolen
PHP Versie	8.1+	8.3+
CPU	4 cores	16+ cores (parallel agents)
RAM	8 GB	32 GB
Redis	6.x	7.x (cluster mode)
Netwerk	100 Mbps	1 Gbps (lage latentie)
LLM API Keys	1x provider	3x providers (diversiteit)

4.2 Environment Variabelen

```
NHM_AGENT1_API_KEY=sk-...
NHM_AGENT2_API_KEY=sk-...
NHM_AGENT3_API_KEY=sk-...
NHM_AGENT1_MODEL=claude-3-5-sonnet-20241022
NHM_AGENT2_MODEL=gpt-4o
NHM_AGENT3_MODEL=gemini-1.5-pro
NHM_MAX_ITERATIONS=10
NHM_CONVERGENCE_THRESHOLD=0.05
NHM_REDIS_HOST=localhost
NHM_CACHE_TTL=3600
```

5. Beveiliging en Rate Limiting

5.1 Beveiligingsmaatregelen

- Prompt Injection Detection: regex en semantische filtering op alle input-prompts.
- API Key Rotation: elke 24 uur automatische sleutelrotatie via Vault.
- TLS 1.3: versleuteling van alle agent-communicatie.
- Input Sanitisatie: HTML/SQL-escaping, lengte-limieten, karakter-whitelisting.
- Rate Limiting: maximaal 100 NHM-verzoeken per minuut per API-key.

5.2 Auditlogging

Alle NHM-aanroepen worden gelogd inclusief: timestamp, agent-scores, iteratiecount, divergence-trace, finale confidence-score en een hash van de prompt (GDPR-compliant).

6. Deployment Gids

6.1 Installatie

```
composer require nhm/core
php artisan nhm:install
php artisan nhm:configure --domain=general --agents=3
```

6.2 Health Check

```
GET /api/v1/nhm/health
{"status":"ok","agents":3,"redis":"connected","embedding_model":"loaded"}
```

6.3 Monitoring

Het NHM exporteert Prometheus-metrics op /metrics:

- nhm_requests_total: Totaal aantal NHM-aanroepen.
- nhm_iterations_histogram: Distributie van iteratiecounts.
- nhm_confidence_gauge: Gemiddelde confidence per tijdvenster.
- nhm_convergence_rate: Percentage verzoeken dat convergeert binnen 5 iteraties.

7. Foutafhandeling

Code	Naam	Beschrijving	Actie
NHM001	AgentTimeoutError	Een agent reageert niet binnen 30 seconden	Retry met backoff
NHM002	NoConvergenceError	Max iteraties bereikt zonder convergentie	Retourneer beste score
NHM003	UnanimousHallucinationError	Alle agents hallucineren identiek	Menselijke review vlag
NHM004	ScoringEngineError	Scoringsalgoritme faalt	Fallback naar enkelvoudig agent
NHM005	EmbeddingModelError	Embedding-service onbeschikbaar	Fallback naar Levenshtein